

HalfApp Agent Action Directives

Date: 2026-05-18

Purpose: convert the HalfApp critical overview into an execution brief for the next coding agent. This is not a research document. It is an action plan for completing the program in the correct order.

Mission

Complete HalfApp by making the active backend and driver-app truthful, auditable, and production-ready before expanding the product surface.

Do not make the app look bigger before the backend can prove what the UI displays.

The core rule:

Every important driver-facing fact must come from durable backend truth, not frontend inference.

Current Product Boundary

Treat only these as active product surfaces:

- backend
- driver-app
- docs/RIDE_LIFECYCLE_CONTRACT.md
- docs/HALFAPP_TRANSPARENCY_ARCHITECTURE.md
- docs/HALFAPP_EXPERT_PROGRAM_CRITICAL_OVERVIEW.md

Treat these as inactive unless explicitly revived, registered, and tested:

- Legacy frontend
- Dormant backend routers not mounted by backend/main.py
- Dormant driver-app diagnostic/demo components
- LocalStorage mock systems
- Demo-only messaging

Non-Negotiable Direction

Do not prioritize:

- Larger UI surface area
- Full rider app
- Admin dashboard
- Payment UI
- Nearest-driver marketing claims
- Fancy maps
- Legacy frontend revival

Prioritize:

- Backend-owned state
- Active driver app alignment
- Audit records
- Database migrations
- Deterministic dispatch behavior
- Financial correctness
- Route proof
- Production hardening

Execution Order

Action 1: Lock The Active Product Boundary

Goal: prevent future work from accidentally using demo or legacy code.

Tasks:

- Confirm backend and driver-app are the only active product path.
- Mark legacy frontend as inactive or archive-only in documentation.
- Confirm unmounted backend routers are not described as active API.
- Identify dormant driver-app components and label them as inactive if they remain.
- Ensure mock mode and guard bypass cannot be silently enabled in production builds.

Done when:

- A new developer can identify the active app path quickly.
- No active doc presents unmounted routes as shipped behavior.
- No production-facing screen depends on browser localStorage as marketplace truth.

Action 2: Make Driver Presence Backend-Owned

Goal: replace local online/offline truth with backend presence truth.

Tasks:

- Add GET /drivers/presence.
- Add PUT /drivers/presence.
- Add POST /drivers/heartbeat.
- Persist driver presence state on the backend.
- Add stale/disconnected state handling.
- Update the cockpit so online/offline state reads from and writes to the backend.
- Keep local UI state only as a cache of the backend response.

Done when:

- Browser refresh does not change driver marketplace truth.
- Backend can explain when a driver was available, offline, paused, stale, or disconnected.
- Dispatch does not depend on stale local browser state.

Action 3: Replace Local Ride Hide With Backend Dismissal

Goal: make Hide for now a real marketplace fact or clearly label it as local-only.

Tasks:

- Add POST /drivers/rides/{ride_id}/hide.
- Add a ride visibility/dismissal record.
- Exclude active hidden rides from that driver's available ride list.
- Store dismissal reason, timestamp, driver ID, ride ID, and expiry/TTL.
- Update cockpit Hide for now to call the backend endpoint.

Done when:

- A hidden ride does not reappear to the same driver until the backend contract allows it.
- The backend can explain when and why the driver hid a ride.
- The UI does not imply a backend decline if only local filtering occurred.

Action 4: Add Dispatch Auditability

Goal: make open-board dispatch explainable.

Tasks:

- Add dispatch policy metadata to available ride responses.
- Return deterministic ordering metadata.
- Record which rides became visible to which drivers.
- Record claim attempted, claim won, and claim lost events.
- Preserve atomic first-claim-wins behavior.
- Return clear 409 Conflict responses when another driver wins.

Done when:

- Backend can explain why a ride appeared to a driver.
- Backend can explain why a claim failed.
- Backend records which dispatch policy made the decision.
- Concurrent claims cannot double-book a ride.

Action 5: Add Marketplace Ledger Events

Status: implemented for the active backend ride/dispatch spine.

Goal: create an append-only audit trail for important marketplace facts.

Tasks:

- Add marketplace_ledger_events.
- Write events for:
 - ride created, - ride visible, - ride hidden, - claim attempted, - claim won, - claim lost, - ride declined/released, - ride cancelled, - ride completed, - earnings calculated.
- Include correlation IDs where practical.

- Include idempotency keys for critical writes where practical.
- Add tests proving event creation.

Done when:

- Important marketplace state changes are not represented only as mutable fields on rides.
- Future driver/admin audit views can be built from backend events.

Current implementation:

- marketplace_ledger_events is append-only at the database layer.
- Event rows include idempotency_key, correlation_id, previous_event_hash, and event_hash.
- Active ride creation, visibility, hide, claim, release, cancellation, completion, presence, and earning-calculation paths write events.

Action 6: Add Database Migration Discipline

Goal: stop relying on startup schema mutation as the long-term schema strategy.

Tasks:

- Add migration tooling, preferably Alembic for the current FastAPI/SQLAlchemy stack.
- Create initial migrations for current schema.
- Move future schema changes into migrations.
- Add foreign keys and indexes where safe and appropriate.
- Document local development migration commands.

Done when:

- Schema changes are repeatable and reviewable.
- Production schema evolution does not depend on create_all.
- Tests can run against a predictable migrated schema.

Action 7: Add Financial Ledger Foundation

Goal: separate money truth from earnings summaries.

Tasks:

- Store money as integer cents, not floating-point values.

- Add financial records for quote, final fare, platform fee, driver earning, adjustment, refund, and payout.
- Track rate-card ID and calculation basis.
- Keep GET /drivers/earnings as a projection over ledger records.
- Add tests for rounding, adjustment, refund, and payout projection behavior.

Done when:

- Driver earnings are explainable down to the cent.
- Platform retained amounts are explainable.
- Adjustments and refunds append new records instead of overwriting history.

Action 8: Add Route Snapshot Foundation

Goal: prevent the UI from inventing distance, route, or ETA truth.

Tasks:

- Add a route snapshot model.
- Add a routing service interface.
- Store provider, engine version, routing profile, origin, destination, distance, duration, calculation timestamp, and geometry hash.
- Link fare quotes and dispatch previews to route snapshots when route data is used.
- Update UI copy so missing route data is shown as unavailable, not guessed.

Done when:

- Any real map, ETA, distance, or route claim points to backend route data.
- Missing route facts are not replaced by frontend guesses.

Action 9: Production Hardening

Goal: make the active MVP deployable without local-development shortcuts.

Tasks:

- Replace development SECRET_KEY.
- Restrict CORS to authorized origins.
- Define token refresh and revocation policy.
- Add structured JSON logging.

- Add request/correlation IDs.
- Validate transaction behavior on the selected production database.
- Add CI checks for backend tests, driver-app build, and critical E2E tests.
- Ensure mock mode and auth bypass are blocked in production builds.

Done when:

- The MVP can deploy without depending on local shortcuts.
- Critical behavior is observable in logs.
- Production configuration cannot silently enable mock or bypass behavior.

First Implementation Slice

Start here:

1. Add backend-owned driver presence.
2. Add backend-backed ride hide/dismissal.
3. Add visibility records.
4. Update the active cockpit to use those backend facts.
5. Add tests proving refresh/reload does not change marketplace truth.

This is the best first slice because it directly removes local UI truth from the driver cockpit without requiring the full financial ledger, route engine, or new marketplace surface.

Agent Operating Rules

- Work only through active backend and driver-app unless explicitly instructed otherwise.
- Do not revive legacy frontend as part of this plan.
- Do not present localStorage as marketplace truth.
- Do not make UI claims that the backend cannot prove.
- Do not add complex infrastructure before the current backend contract needs it.
- Prefer incremental backend records and tests over large untested rewrites.
- Keep documentation aligned with implemented behavior after each phase.

Success Definition

HalfApp is moving in the right direction when every important screen can answer:

- Which backend record proves this?
- Which endpoint changed it?
- Which test covers it?
- Which audit event explains it?
- What happens on refresh, retry, conflict, or disconnect?

If those questions cannot be answered, the feature is not complete.